IPsec: IKE (Internet Key Exchange)

Dr. Pinaki Sankar Chatterjee

https://www.youtube.com/watch?v=tapoOQ-MkPU

IPsec: IKE, Internet Key Exchange

- IPsec does not use Public Key Infrastructure and exchanging keys before an IPsec connection is established is a problem.
- IKE solves generation of a symmetric key for a session of IPsec but without PKI man-in-the-middle attack is possible.
- IKE (Internet Key Exchange) creates Security Associations (SA). That is, parties in IKE negotiate keys for the SA.
- SA was a data structure containing keys and other relevant information for the connection.
- IKE is a general purpose key exchange protocol. It is used by IPsec, but it can be used by other protocols who need SAs as well.

IPsec: IKE, Internet Key Exchange

- **General**
- IKE creates SA, refreshes them and deletes them.
- IKE has the following exchanges:
  - Phase one (creation of IKE SA): There are two modes for phase one: main mode or aggressive mode
  - Phase two (creation of IPSec SA): there is only one mode: quick mode
  - Maintenance of IKE SA
  - Negotiation of private Diffie-Hellman groups
- What the last exchange means is that in the phase one there are predefined several ways to use Diffie-Hellman, but one can define own ways also using the last exchange.
- IKE protocol initial message exchanges are not encrypted.
- IKE uses (normally) the UDP port 500.

IPsec: IKE

- **The predefined Diffie-Hellman groups in IKE:**
- (group here means only an agreement of the algorithm)
- 1. MODP group with a 768-bit modulus
- 2. MODP group with a 1024-bit modulus
- 3. ECP group with a 155-bit modulus
- 4. EC2N group with a 185-bit modulus
- 5. MODP group with a 1680-bit modulus

What this means is that you can use discrete logarithm problem noted as MODP and the number p for $A=g^a \bmod p$ must have the defined length. The algorithm family EC2N is a family of elliptic curve cryptoalgorithms. They give good security level with shorter keys and less processing. ECP 155 is about as secure MODP 768, respectively EC2N 185 about as good as MODP 1024.

IPsec: IKE

- In the first part of the IKE exchange, an authentication method is agreed.
- There are five authentication methods
- 1) preshared keys
- 2) digital signature with DSA
- 3) digital signature with RSA
- 4) authentication via exchange of encrypted nonces
- 5) revised method 4)
- This method is agreed via exchange of IKE SA.
- Exchange of IKE SA contains also some secret information.
- The peers generate four secrets: SKEYID, SKEYID_d, SKEYID_a and SKEYID_e.
- Both sides take part in creating the secrets.

IPsec: IKE

- **Generation of the secrets:**
- Each side contributes a cookie (CKY-x) and a nonce (Nx) to SKEYID generation (x=i (initiator) or r (responder).
- A nonce is simply a pseudo-random number, a cookie is generated by taking a hash from some data.
- For preshared key authentication
  - SKEYID=PRF(preshared key, Ni|Nr)
- For signature authentication Diffie-Hellman type g^{xy} is used:
  - SKEYID=PRF(Ni|Nr, g^{xy})
- For encrypted nonce authentication:
  - SKEYID=PRF(hash(Ni|Nr), CKY-i|CKY-r)
- Here | denotes concatenating the data, so Ni|Nr = nonce from initiator + nonce from responder. PRF is a result of a hash function, usually HMAC.

IPsec: IKE

- **All other secrets are derived from SKEYID:**
- SKEYID_d=PRF(SKEYID, g^{xy}|CKY-i|CKY-r|0)
- SKEYID_a=PRF(SKEYID, SKEYID_d|g^{xy}|CKY-i|CKY-r|1)
- SKEYID_e=PRF(SKEYID, SKEYID_a|g^{xy}|CKY-i|CKY-r|2)
- **Why all these secrets?**
- SKEYID_d is used for deriving keying data for IPSec
- SKEYID_a is used for integrity and data source authentication
- SKEYID_e is used to encrypt IKE messages.
- Different keys must be used for security purposes. Because of the hash function PRF, the original secret SKEYID cannot be calculated from the derived secrets.
- Why so many options in IKE (remember, many options were one reason OSI failed to gain popularity) ?

IPsec: IKE

- **cookie exchange**
- IKE uses the following cookie generation method:
- a cookie is the result of hashing a unique identifier of the peer (peer's IP address, port and protocol), a secret known only to the generator of the cookie, and a time stamp.
- The initiator generates a cookie, sets the responder cookie to zero and sends to the responder.
- The responder generates a responder cookie, copies the initiator cookie to the message and sends it to the initiator.
- The initiator can easily check that the initiator cookie is to one it generated and that the peer's addresses match.
- Only if the cookie matches, check of signatures etc. are made.
- Consider this: if you need to check the signatures (strong method), the cookie method must be weak. How can a weak method protect a strong method? Thus, there must be an attack leading the parties to the computationally expensive Diffie-Hellman and DoS. Try the cookie method: initiator=attacker

IPsec: IKE

- **Phase one, normal mode**
- using preshared key authentication
- Initiator Responder

Header, SA	$\rightleftarrows$	Header, SA
Header, KE, Nonce	$\rightleftarrows$	Header, KE, Nonce
Header, IDi, Hash	$\rightleftarrows$	Header, IDi, Hash

The normal mode has an exchange of six messages, several versions of the phase one normal mode exist. SA=Security Association, KE=Key Exchange, Nonce=random number, IDi= identity of the peer.

IPsec: IKE

- **Phase one of normal mode**
- using public key exchanges:
- Initiator Responder

Header, SA	$\rightleftarrows$	Header, SA
Header, KE, Ni [,Cert_Req]	$\rightleftarrows$	Header, KE, Ni [,Cert_Req]
Header, IDi, [Cert,] Signature	$\rightleftarrows$	Header, IDi, [Cert,] Signature

In this variant optional payloads are bracketed. In the optional features a certificate can be requested (Cert_Req) and then it is returned in Cert. Ni=Nonce i.

IPsec: IKE

- **Phase one of normal mode**
- the standard method using public key exchanges:
- Initiator Responder

Header, SA	$\rightleftarrows$	Header, SA
Header, KE, {IDi}pub_r, {Ni}pub_r	$\rightleftarrows$	Header, KE, {IDi}pub_i, {Ni}pub_r
Header, Hash	$\rightleftarrows$	Header, Hash

In this variant {something}pub_x means something encrypted with the public key of x=i (initiator) or r (responder). Ni is nonce.

IPsec: IKE

- **Policy negotiation**
- After IKE SA is agreed, IKE will negotiate of the policy.
- Policy is something like: authenticate everything and if possible encrypt it, and if possible also compress it.
- For each operation there may be several algorithms.
- SA payload can contain several proposals for protocols and exact algorithms (transforms).
- Policy negotiation works so that the initiator proposes some algorithms and the responder removes from the list what it does not want to use.
- Negotiating compression is also included in IKE since it is not good to try to compress encrypted data (it will not compress, it is random), therefore link layer compression like in PPP will not work with IPsec. One (but not efficient) way is to compress each IP packet on IPsec layer before encryptation with PCP.

IPsec: IKE

Example proposal for SA: the offer maker proposes in the given order the following choices.

Proposal 1: AH

- Transform 1: HMAC-SHA
- Transform 2: HMAC-MD5

Proposal 2: ESP

- Transform 1: 3DES with HMAC-SHA
- Transform 2: 3DES with HMAC-MD5
- Transform 3: DES with HMAC-SHA
- Transform 4: DES with HMAC-MD5

Proposal 3: PCP (compression before encryptation)

- Transform 1: LZS (one header compression algorithm)
- Transform 2: Deflate (another header compression)

IPsec: IKE

- **Phase one: aggressive mode**
- Aggressive mode is more simple than the normal mode. In the aggressive mode there are only three messages exchanged.
- The initiator offers a list of protection suites, his Diffie-Hellman public key value, his nonce and his identity.
- The responder replies with a selected protection suite, his Diffie-Hellman public value, his nonce, his identity, and authentication payload, like a signature.
- The initiator responds with authentication payload.
- There is no chance to negotiate as much in this case as in the normal mode.
- The method suits well for connecting to own site from a remote site as then it is known in advance what kind of authentication the other side supports.

IPsec: IKE

- **Phase two: quick mode**
- Phase two of IKE creates IPsec SA. Since IKE can be used for other protocols than IPsec, like the routing protocols RIPv2 and OSPF, IKE SA is not directly IPsec SA.
- IKE SA protects the quick mode by encrypting messages and authenticating them. Authentication comes from use of PRF (the HMAC hash function)
- The quick mode creates keys for IPSec association.
- Many quick modes can be made using the same IKE SA, therefore a message ID (M-ID) is used to identify the IPSec SA. Nonces are added to prevent replay of the same messages by an attacker.
- The quick mode has more details, but the following figure gives the general view of the protocol.

IPsec: IKE

- **Quick mode exchange**
- Initiator Responder

Header, HASH1, SA, Ni
[, KE][, IDci, IDcr] →

← Header, HASH2, SA, Nr
[, KE] [, IDci, IDcr]

Header, HASH3 →

HASH1=PRF(SKEYID_a, M-ID | SA | Ni [| KE] [| IDci | IDcr])

HASH2=PRF(SKEYID_a, M-ID | Ni | SA [| KE] [| IDci | IDcr])

HASH3=PRF(SKEYID_a, 0 | M-ID | Ni | Nr)

H_1, SA → KEA, G, P, ENC/DEC, g → generator

(D-H) group. prime no. Hash.

S ⟶ R

H_2, SA = G, P, ENC/DEC, KEA, g

hash

ACK

Correctly received.

H_3, KEA (G, P, Nonce) N_1

$N_1 \Rightarrow g^{\alpha} \bmod p$

$\alpha \rightarrow$ Secret.

N_1 ⟶ H_4, KEA (G, P, Nonce) N_2

$N_2 = g^{\beta} \bmod p$

$\beta \rightarrow$ Secret.

N_2

H_5, ID_1, ID_2, Hash

sender receiver

$H(K_1)$ ⟶ verify

$N_1^{\beta} \bmod p$

$= (g^{\alpha} \bmod p)^{\beta} \bmod p$

$= g^{\alpha\beta} \bmod p$

$= K_1^*$

$H(K_1^*) \overset{?}{=} H(K_1)$

$H(g^{\alpha\beta} \bmod p)$ ⇐ $(N_2^{\alpha} \bmod p)$

$K_1 = (g^{\beta} \bmod p)^{\alpha} \bmod p$

$= g^{\alpha\beta} \bmod p$

ACK

H_6, ID_2, ID_1, $H(K_1^*)$

verify